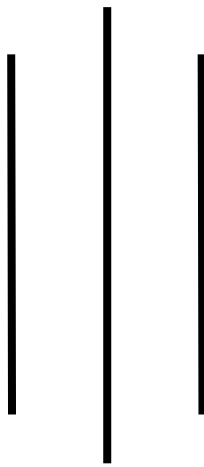


TRIBHUVAN UNIVERSITY

PATAN MULTIPLE CAMPUS

PATANDHOKA, LALITPUR



NET CENTRIC COMPUTING (BIT 351)

SUBMITTED BY:

NAME: SURESH DAHAL

ROLL: 23

CLASS: BIT-III/II

DATE: 2083/05/15

SUBMITTED TO:

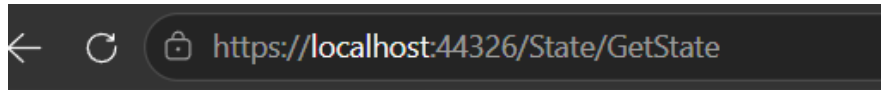
SUBASH SIWA

CHECKED BY

UNIT-6

1. Implementing Server-Side State Management in ASP.NET Core.

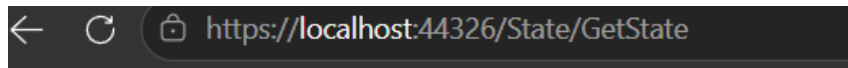
OUTPUT:



Server-Side State Values

- **Username (Session):** Suresh Dahal
- **Message (TempData):** Successfully saved!
- **Cached Time (Memory Cache):** 8/23/2026 1:10:55 PM

First load



Server-Side State Values

- **Username (Session):** Suresh Dahal
- **Message (TempData):**
- **Cached Time (Memory Cache):** 8/23/2026 1:10:55 PM

After refresh

2. Implementing Client-Side State Management

PROGRAM:

1. Create MVC Application

```
dotnet new mvc -n StateDemo
```

```
cd StateDemo
```

2. Controller

```
using Microsoft.AspNetCore.Mvc;

namespace StateMgmtClientSide.Controllers;

public class ClientStateController : Controller
{
    [HttpGet]
    public IActionResult Index(string? name, int? age)
    {
        ViewBag.Name = name;
        ViewBag.Age = age;
        ViewBag.Theme = Request.Cookies["Theme"] ?? "Not Set";

        return View();
    }

    [HttpGet]
    public IActionResult SaveCookie(string theme)
    {
        Response.Cookies.Append("Theme", theme);

        return RedirectToAction("Index");
    }

    [HttpGet]
    public IActionResult DeleteCookie()
    {
        Response.Cookies.Delete("Theme");

        return RedirectToAction("Index");
    }

    [HttpPost]
    public IActionResult SubmitForm(int userId, string name)
    {
        ViewBag.Result =
            $"User ID: {userId}, Name: {name}";

        ViewBag.Theme =
            Request.Cookies["Theme"] ?? "Not Set";

        return View("Index");
    }
}
```

3. View

```
<h2>Client-Side State Management</h2>
<hr />
<h3>1. Cookie</h3>

<a href="/ClientState/SaveCookie?theme=Dark">
    Save Cookie
</a>

<p>
    Current Theme:
    <b>@ViewBag.Theme</b>
</p>

<a href="/ClientState/DeleteCookie">
    Delete Cookie
</a>

<hr />

<h3>2. Query String</h3>

<p>
    Name: <b>@ViewBag.Name</b>
    <br />
    Age: <b>@ViewBag.Age</b>
</p>

<a href="/ClientState?name=Suresh&age=26">
    Send Name and Age using Query String
</a>
<hr />
<h3>3. Hidden Field</h3>

<form action="/ClientState/SubmitForm" method="post">

    <input type="hidden"
        name="userId"
        value="101" />

    <label>Name:</label>

    <input type="text"
        name="name"
        placeholder="Enter name" />

    <button type="submit">
        Submit
    </button>

</form>

@if (ViewBag.Result != null)
{
    <p>
        <b>@ViewBag.Result</b>
    </p>
}
```

OUTPUT:

1. Cookie

[Save Cookie](#)

Current Theme: **Not Set**

[Delete Cookie](#)

Before setting cookie

1. Cookie

[Save Cookie](#)

Current Theme: **Dark**

[Delete Cookie](#)

after setting cookie



2. Query String

Name: **Suresh**

Age: **26**

[Send Name and Age using Query String](#)

Query String

3. Hidden Field

Name:

User ID: 101, Name: Suresh

Hidden Field

UNIT – 7

1. Client-Side Dynamic Web Development with jQuery and AJAX

THEORY:

jQuery is a JavaScript library used for simplifying client-side web development. It provides easy methods for selecting HTML elements, handling events, manipulating the DOM, validating forms, and communicating with servers using AJAX.

AJAX (Asynchronous JavaScript and XML) allows a web page to communicate with a server without refreshing the entire page. In an ASP.NET Core application, jQuery can send AJAX requests to Web API REST endpoints and receive data, usually in JSON format.

Advantages of jQuery

- Simple and easy-to-use syntax.
- Simplifies DOM manipulation.
- Provides convenient event handling.
- Supports AJAX communication.
- Allows dynamic page updates without refreshing the browser.

PROGRAM

1. ASP.NET Core Web API

Create the project:

```
dotnet new webapi -n StudentApi
cd StudentApi
```

Create Models/Student.cs:

```
namespace StudentApi.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; } = "";
        public string Email { get; set; } = "";
    }
}
```

Create Controllers/StudentsController.cs:

```
using Microsoft.AspNetCore.Mvc;
using StudentApi.Models;

namespace StudentApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class StudentsController : ControllerBase
```

```

{
    private static List<Student> students = new()
    {
        new Student
        {
            Id = 1,
            Name = "Suresh",
            Email = "suresh@gmail.com"
        },
        new Student
        {
            Id = 2,
            Name = "Luna",
            Email = "Luna@gmail.com"
        }
    };

    [HttpGet]
    public IActionResult GetStudents()
    {
        return Ok(students);
    }

    [HttpPost]
    public IActionResult AddStudent(Student student)
    {
        student.Id = students.Count + 1;
        students.Add(student);

        return Ok(student);
    }
}
}

```

Run the API:

dotnet run

2. Client-side HTML, CSS and jQuery

Create index.html:

```

<!DOCTYPE html>
<html>
<head>
    <title>Student Management</title>
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
    <style>
        body {
            font-family: Arial;

```

```

        margin: 40px;
    }
    input {
        padding: 8px;
        margin: 5px;
    }
    button {
        padding: 8px 15px;
    }
    li {
        margin: 8px;
    }
    .error {
        color: red;
        font-size: 14px;
    }
</style>
</head>
<body>
    <h1>Student Management</h1>
    <form id="studentForm">
        <input type="text" id="name" placeholder="Enter name">
        <span id="nameError" class="error"></span>
        <br>
        <input type="text" id="email" placeholder="Enter email">
        <span id="emailError" class="error"></span>
        <br>
        <button type="submit">Add Student</button>
    </form>
    <br>
    <button id="loadStudents">Load Students</button>
    <ul id="studentList"></ul>

```

```

<script>
    $("#loadStudents").click(function () {
        $.ajax({
            url: "https://localhost:44321/api/students",
            type: "GET",
            success: function (students) {
                $("#studentList").empty();
                $.each(students, function (index, student) {
                    $("#studentList").append(
                        "<li>" + student.id + " - " +
                        student.name + " - " +
                        student.email +
                        "</li>"
                    );
                });
            },
            error: function () {
                alert("Error loading students.");
            }
        });
    });

    $("#studentForm").submit(function (event) {
        event.preventDefault();
        $("#nameError").text("");
        $("#emailError").text("");
        var name = $("#name").val().trim();
        var email = $("#email").val().trim();
        var valid = true;
        if (name === "") {
            $("#nameError").text("Name is required.");
            valid = false;
        }
    });

```

```

var emailPattern =
    /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (email === "") {
    $("#emailError").text("Email is required.");
    valid = false;
}
else if (!emailPattern.test(email)) {
    $("#emailError").text("Enter a valid email.");
    valid = false;
}
if (!valid) return;
var student = {
    name: name,
    email: email
};
$.ajax({
    url: "https://localhost:44321/api/students",
    type: "POST",
    contentType: "application/json",
    data: JSON.stringify(student),
    success: function (student) {
        $("#studentList").append(
            "<li>" + student.id + " - " +
            student.name + " - " +
            student.email +
            "</li>"
        );
        $("#studentForm")[0].reset();
    },
    error: function () {
        alert("Error adding student.");
    }
}

```

```
        });  
    });  
</script>  
</body>  
</html>
```

OUTPUT:

Student Management

Suresh Dahal	su@resh.com	Add Student
Load Students		

Add students

Student Management

Enter name	Enter email	Add Student
Load Students		

- 1 - Suresh - suresh@gmail.com
- 2 - Luna - luna@gmail.com
- 3 - Govinda - govi@nda.com
- 4 - Suresh Dahal - su@resh.com

Load all students

Student Management

hari	
hari	Enter a valid email.
Add Student	

Client-side validation using jQuery

2. jQuery vs. Modern Frontend Frameworks

THEORY:

- **jQuery:** An imperative DOM manipulation library. You manually select elements and explicitly tell the browser how to update the DOM step-by-step. It is lightweight and easy to drop into legacy or small-scale multi-page applications.
- **Modern Frameworks:** Component-based, declarative UI libraries/frameworks designed for building large-scale Single Page Applications.
- **Key Comparisons:**
 - **Architecture:** jQuery handles individual elements imperatively; modern frameworks use reusable components with a reactive Component Lifecycle.
 - **Data Binding:** jQuery requires manual reading/writing of values between UI inputs and variables. Modern frameworks feature Two-Way (Angular/Vue) or One-Way Data Binding with Virtual DOM (React) that synchronizes state and UI automatically.
 - **State Management & Scaling:** jQuery code can become difficult to maintain as an app grows. Modern frameworks provide structured state management ecosystems optimized for complex enterprise applications.

Practical Comparison Table

Feature / Aspect	jQuery	Modern Frameworks
Approach	Imperative	Declarative
Structure	Script-based manipulation of global DOM elements	Component-based, modular architecture
Data Synchronization	Manual (e.g., \$('#input').val())	Automatic via Data Binding & State Management
Best Used For	Small apps, widget enhancements, legacy apps	Large-scale Single Page Applications

Example

jQuery:

```
$("#message").text("Hello World");
```

React:

```
function App() {  
  return <h1>Hello World</h1>;  
}
```

In jQuery, the developer directly selects and modifies DOM elements. In React, the interface is organized into reusable components and is updated based on application state.

UNIT – 8:

1. Implementing Authentication Using ASP.NET Core Identity

THEORY

ASP.NET Core Identity is a membership system that provides authentication and user-management functionality for ASP.NET Core applications. It manages users, passwords, roles, claims, login, logout, and password-related operations.

The basic authentication process is:

User



Login/Register



ASP.NET Core Identity



Identity Database



Authentication Cookie



Authorized Application

PRACTICAL IMPLEMENTATION

Step 1: Create MVC Application

```
dotnet new mvc -n IdentityDemo
cd IdentityDemo
```

Install Identity and Entity Framework Core packages:

```
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

Step 2: Create Identity DbContext

Create Data/AppDbContext.cs:

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
```

```

namespace IdentityDemo.Data;

public class AppDbContext
    : IdentityDbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }
}

```

Step 3: Configure Identity

In Program.cs:

```

using IdentityDemo.Data;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(
        builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddDefaultIdentity<IdentityUser>(options =>
{
    options.SignIn.RequireConfirmedAccount = false;
    options.Password.RequireDigit = true;
    options.Password.RequiredLength = 6;
})
.AddEntityFrameworkStores<AppDbContext>();

builder.Services.AddControllersWithViews();

var app = builder.Build();

```

```
app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.MapRazorPages();
app.Run();
```

Step 4: Configure Database

Add the following to appsettings.json:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=identity.db"
  }
}
```

Create the Identity database:

```
dotnet ef migrations add InitialIdentity
```

```
dotnet ef database update
```

This creates database and Identity tables.

Step 5: Create Authentication Controller

Create Controllers/AccountController.cs:

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
```

```

namespace IdentityDemo.Controllers;

public class AccountController : Controller
{
    private readonly UserManager<IdentityUser> _users;
    private readonly SignInManager<IdentityUser> _signIn;

    public AccountController(
        UserManager<IdentityUser> users,
        SignInManager<IdentityUser> signIn)
    {
        _users = users;
        _signIn = signIn;
    }

    [HttpGet]
    public IActionResult Register() => View();

    [HttpPost]
    public async Task<IActionResult> Register(string email, string password)
    {
        var user = new IdentityUser { UserName = email, Email = email };

        var result = await _users.CreateAsync(user, password);

        if (result.Succeeded)
        {
            await _signIn.SignInAsync(user, false);
            return RedirectToAction("Index", "Home");
        }

        ViewBag.Error = "Registration failed.";
    }
}

```

```

        return View();
    }

    [HttpGet]
    public IActionResult Login() => View();

    [HttpPost]
    public async Task<IActionResult> Login(string email, string password)
    {
        var result = await _signIn.PasswordSignInAsync(
            email, password, false, false);

        if (result.Succeeded)
            return RedirectToAction("Index", "Home");

        ViewBag.Error = "Invalid email or password.";
        return View();
    }

    public async Task<IActionResult> Logout()
    {
        await _signIn.SignOutAsync();
        return RedirectToAction("Index", "Home");
    }
}

```

Step 6: Register View

Create Views/Account/Register.cshtml:

```

<h2>Register</h2>

<form method="post">
    <input name="email" type="email" placeholder="Email" required />
    <input name="password" type="password"

```

```

        placeholder="Password" required />
        <button type="submit">Register</button>
    </form>

```

```

@if (ViewBag.Error != null) {
    <p>@ViewBag.Error</p>
}

```

Step 7: Login View

Create Views/Account/Login.cshtml:

```

<h2>Login</h2>
<form method="post">
    <input name="email" type="email" placeholder="Email" required />
    <input name="password" type="password"
        placeholder="Password" required />
    <button type="submit">Login</button>
</form>
@if (ViewBag.Error != null) {
    <p>@ViewBag.Error</p>
}

```

Step 8: Protect a Controller

Modify HomeController.cs:

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
public class HomeController : Controller
{
    [AllowAnonymous]
    public IActionResult Index() {
        return View();
    }
}

```

```

[Authorize]

public IActionResult Dashboard() {
    return View();
}
}

```

[AllowAnonymous] allows anyone to access the action.

[Authorize] allows only authenticated users to access the action.

OUTPUT:

Add Packages

```

D:\Online Class\suresh\AuthCoreIdentity\AuthCoreIdentity>dotnet add package Microsoft.EntityFrameworkCore.Tools
info : X.509 certificate chain validation will use the default trust store selected by .NET for code signing.
info : X.509 certificate chain validation will use the default trust store selected by .NET for timestamping.
info : Adding PackageReference for package 'Microsoft.EntityFrameworkCore.Tools' into project 'D:\Online Class\s
uresh\AuthCoreIdentity\AuthCoreIdentity\AuthCoreIdentity.csproj'.
info : GET https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.tools/index.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.tools/index.json 543
ms

```

dotnet ef migrations add InitialIdentity

```

D:\Online Class\suresh\AuthCoreIdentity\AuthCoreIdentity>dotnet ef migrations add InitialIdentity
Build started...
Build succeeded.
Done. To undo this action, use 'ef migrations remove'

```

dotnet ef database update

```

D:\Online Class\suresh\AuthCoreIdentity\AuthCoreIdentity>dotnet ef database update
Build started...
Build succeeded.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (22ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      PRAGMA journal_mode = 'wal';
info: Microsoft.EntityFrameworkCore.Migrations[20411]

```

Registration

← ↻ 🔒 https://localhost:44357/account/register

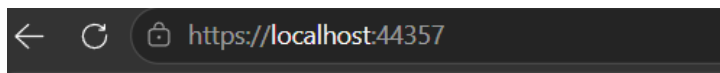
Register

Login

← ↻ 🔒 https://localhost:44357/account/register

Register

Successful registration redirects to home page

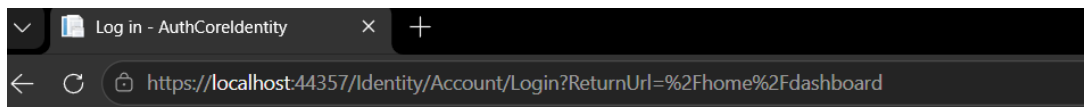


Welcome to the Home Page

Protected page

/Home/Dashboard

Without authentication:



AuthCoreIdentity

Log in

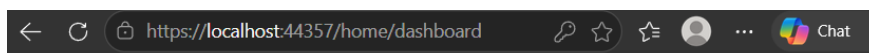
Use a local account to log in.

☐ Remember me?

[Forgot your password?](#)

[Register as a new user](#)

After login:



This is protected dashboard page. You must be logged in to view this page.

2. Implementing Authorization Using Roles, Claims, and Policies

Theory

Authorization determines what authenticated user is allowed to access. ASP.NET Core supports role-based, claim-based, and policy-based authorization.

- **Role:** It groups users according to their responsibilities, such as Admin, Faculty, or Student.
- **Claim:** It is a piece of information about a user, such as department or permission.
- **Policy:** It is a rule that determines whether a user can access a resource based on roles, claims, or other requirements.

ASP.NET Core provides attributes such as:

- [Authorize] allows only authenticated users.
- [AllowAnonymous] allows everyone, including unauthenticated users.
- [Authorize(Roles = "Role")] restricts access to a specific role.
- [Authorize(Policy = "PolicyName")] restricts access according to a policy.

Practical Implementation

1. Configure Roles

Program.cs

```
using AuthCoreIdentity.Data;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(
        builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddDefaultIdentity<IdentityUser>()
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<AppDbContext>();

builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("FacultyOnly",
        policy => policy.RequireClaim("Department", "IT"));
});

builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseHttpsRedirection();
```

```

app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.MapRazorPages();
using (var scope = app.Services.CreateScope())
{
    await SeedData.Initialize(scope.ServiceProvider);
}

app.Run();

```

2. Creating Roles

Create Data/SeedData.cs:

```

using Microsoft.AspNetCore.Identity;

namespace AuthCoreIdentity.Data;

public static class SeedData
{
    public static async Task Initialize(
        IServiceProvider services)
    {
        var roleManager =
            services.GetRequiredService<RoleManager<IdentityRole>>();

        foreach (var role in new[] { "Admin", "Faculty", "Student" })
        {
            if (!await roleManager.RoleExistsAsync(role))
                await roleManager.CreateAsync(new IdentityRole(role));
        }
    }
}

```

The three roles are now created:

Admin, Faculty, Student

3. Assigning a Role to a User

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using System.Security.Claims;

namespace AuthCoreIdentity.Controllers;

public class AccountController : Controller
{

```

```

private readonly UserManager<IdentityUser> _users;
private readonly SignInManager<IdentityUser> _signIn;
public AccountController(
    UserManager<IdentityUser> users,
    SignInManager<IdentityUser> signIn)
{
    _users = users;
    _signIn = signIn;
}

[HttpPost]
public async Task<IActionResult> Register(string email, string password)
{
    var user = new IdentityUser
    {
        UserName = email,
        Email = email
    };

    var result = await _users.CreateAsync(user, password);

    if (!result.Succeeded)
    {
        ViewBag.Error = string.Join(
            "<br>",
            result.Errors.Select(e => e.Description));

        return View();
    }

    // Add Admin role
    var roleResult =
        await _users.AddToRoleAsync(user, "Admin");

    if (!roleResult.Succeeded)
    {
        ViewBag.Error =
            "Role error: " +
            string.Join(
                "<br>",
                roleResult.Errors.Select(e => e.Description));

        return View();
    }

    // Add Department claim
    var claimResult =
        await _users.AddClaimAsync(
            user,
            new Claim("Department", "IT"));

    if (!claimResult.Succeeded)
    {
        ViewBag.Error =
            "Claim error: " +
            string.Join(
                "<br>",
                claimResult.Errors.Select(e => e.Description));
    }
}

```

```

        return View();
    }

    await _signIn.SignInAsync(user, false);

    return RedirectToAction("Index", "Home");
}

[HttpGet]
public IActionResult Login() => View();

[HttpPost]
public async Task<IActionResult> Login(string email, string password)
{
    var result = await _signIn.PasswordSignInAsync(
        email, password, false, false);

    if (result.Succeeded)
        return RedirectToAction("Index", "Home");

    ViewBag.Error = "Invalid email or password.";
    return View();
}
}

```

4. Authorization

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace AuthCoreIdentity.Controllers;

[Authorize]
public class AdminController : Controller
{
    [Authorize(Roles = "Admin")]
    public IActionResult Index()
    {
        return Content("Welcome Admin Suresh");
    }

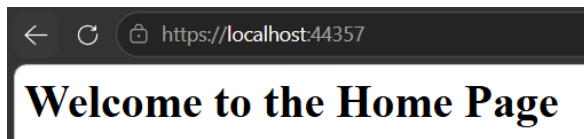
    [Authorize(Policy = "FacultyOnly")]
    public IActionResult Faculty()
    {
        return Content("Faculty IT Area");
    }

    [AllowAnonymous]
    public IActionResult Public()
    {
        return Content("Public Area");
    }
}

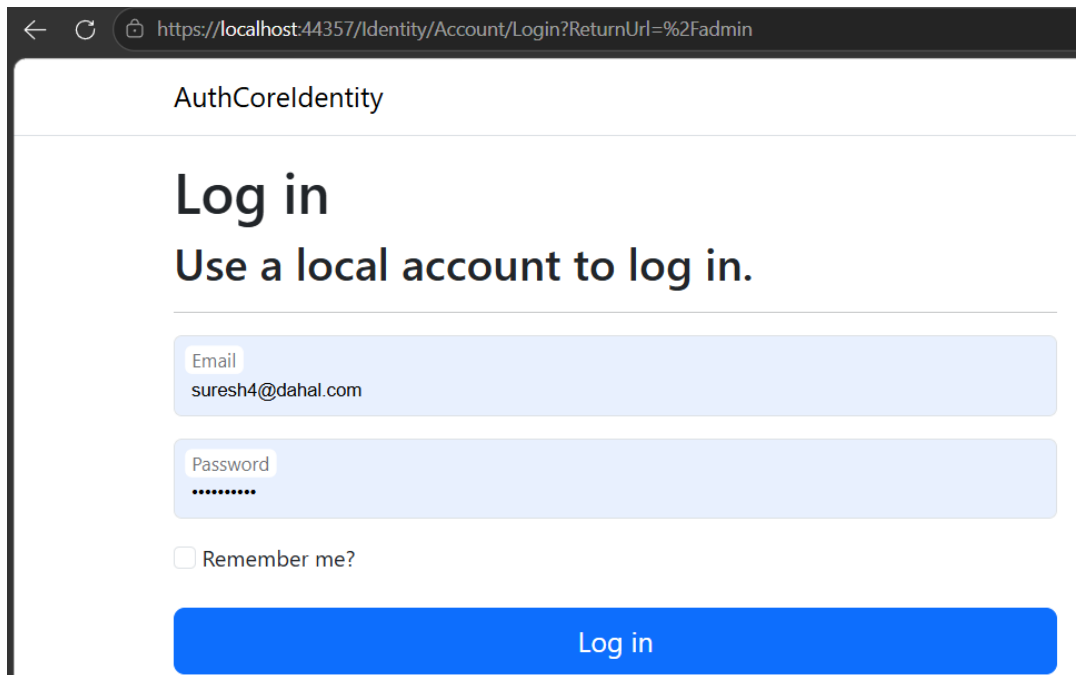
```

OUTPUT:

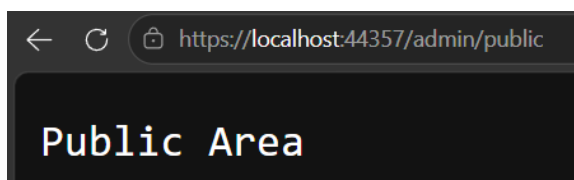
Home page



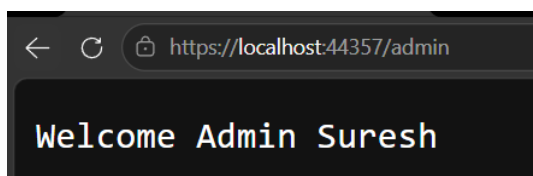
Visiting /admin without login redirects to login page



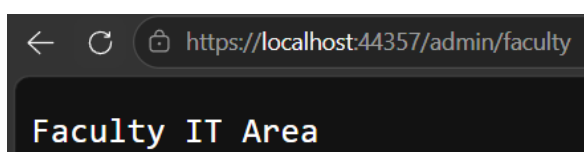
[AllowAnonymous] allows visiting public endpoint inside admin controller



Admin route, protected by role



Faculty route, protected by policy



3. Common Web Application Vulnerabilities and Prevention

THEORY:

ASP.NET Core provides several security features for protecting web applications from unauthorized access and common attacks. These include authentication, authorization, input validation, output encoding, anti-forgery protection, secure database access, and safe URL handling.

Common web application vulnerabilities include Cross-Site Scripting (XSS), SQL Injection, Cross-Site Request Forgery (CSRF), and Open Redirect attacks.

1. Cross-Site Scripting (XSS)

XSS occurs when an attacker injects malicious script into a web page through user input. If the application displays the input without proper encoding, the script may be executed in the user's browser.

For example: `<script>alert('XSS')</script>`

Prevention

- Validate user input.
- Encode output before displaying it.
- Avoid `Html.Raw()` with untrusted data.
- Sanitize HTML when accepting HTML content is necessary.

ASP.NET Core Razor automatically HTML-encodes normal output, providing protection against many XSS attacks.

2. SQL Injection

SQL Injection occurs when user input is directly included in an SQL query. An attacker may manipulate the input to change the intended SQL command.

An example of unsafe query:

```
"SELECT * FROM Users WHERE Username = " + username + ""
```

Prevention

- Use Entity Framework Core.
- Use parameterized queries.
- Never concatenate untrusted input directly into SQL.
- Validate input where appropriate.

3. Cross-Site Request Forgery (CSRF)

CSRF occurs when an attacker causes a user's browser to send an unwanted request to a website where the user is already authenticated.

ASP.NET Core MVC provides anti-forgery tokens to ensure that a request originated from the legitimate application.

Prevention

Use:

```
[ValidateAntiForgeryToken]
```

on sensitive POST actions.

ASP.NET Core's Form Tag Helper can automatically include the anti-forgery token in forms.

4. Open Redirect

An Open Redirect vulnerability occurs when an application redirects a user to a URL supplied by the user without checking whether the destination is trusted.

For example:

```
/login?returnUrl=https://malicious-site.com
```

An attacker could use such a link to make a trusted application appear to redirect users to a malicious website.

Prevention

Use:

```
Url.IsLocalUrl(returnUrl)
```

to verify that the destination is local, and use:

```
LocalRedirect(returnUrl)
```

for safe local redirection.

PROGRAM:

1. Install ADO.NET SQLite Package

```
dotnet add package Microsoft.Data.Sqlite
```

2. Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Security}/{action=Index}/{id?}");

app.Run();
```

3. SecurityController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.Data.Sqlite;

namespace SecurityDemo.Controllers;

public class SecurityController : Controller
{
    private readonly string connectionString =
        "Data Source=security.db";

    private void InitializeDatabase()
    {
        using var con = new SqliteConnection(connectionString);
        con.Open();

        string sql = """
        CREATE TABLE IF NOT EXISTS Users (
            Id INTEGER PRIMARY KEY AUTOINCREMENT,
            Username TEXT NOT NULL,
            Password TEXT NOT NULL
        );

        INSERT OR IGNORE INTO Users
        (Id, Username, Password)
        VALUES (1, 'admin', '1234');
        """;

        using var cmd = new SqliteCommand(sql, con);
        cmd.ExecuteNonQuery();
    }

    public SecurityController()
    {
        InitializeDatabase();
    }

    public IActionResult Index()
```

```

{
    return View();
}

// ----- XSS -----

[HttpPost]
public IActionResult Xss(string name)
{
    if (string.IsNullOrEmpty(name))
    {
        ViewBag.XssResult = "Name is required.";
        return View("Index");
    }

    ViewBag.XssResult = name;

    return View("Index");
}

// ----- SQL INJECTION -----

// Vulnerable example
[HttpPost]
public IActionResult UnsafeLogin(
    string username, string password)
{
    using var con = new SqlConnection(connectionString);

    string sql =
        $"SELECT COUNT(*) FROM Users " +
        $"WHERE Username='{username}' " +
        $"AND Password='{password}'";

    using var cmd = new SqlCommand(sql, con);

    con.Open();

    int count = Convert.ToInt32(cmd.ExecuteScalar());

    ViewBag.SqlResult =
        count > 0
        ? "Login successful (unsafe query)."
        : "Invalid login.";

    return View("Index");
}

// Secure parameterized query
[HttpPost]
public IActionResult SafeLogin(
    string username, string password)
{
    using var con = new SqlConnection(connectionString);

    string sql =
        "SELECT COUNT(*) FROM Users " +
        "WHERE Username=@username " +
        "AND Password=@password";

```

```

        using var cmd = new SqliteCommand(sql, con);

        cmd.Parameters.AddWithValue(
            "@username", username);

        cmd.Parameters.AddWithValue(
            "@password", password);

        con.Open();

        int count = Convert.ToInt32(cmd.ExecuteScalar());

        ViewBag.SqlResult =
            count > 0
                ? "Login successful (parameterized query)."
                : "Invalid login.";

        return View("Index");
    }

    // ----- CSRF -----

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Submit(string message)
    {
        ViewBag.CsrfResult =
            "Request accepted. Anti-forgery token is valid.";

        return View("Index");
    }

    // ----- OPEN REDIRECT -----

    // Vulnerable
    public IActionResult UnsafeRedirect(string url)
    {
        return Redirect(url);
    }

    // Secure
    public IActionResult SafeRedirect(string url)
    {
        if (Url.IsLocalUrl(url))
            return LocalRedirect(url);

        ViewBag.RedirectResult =
            "External URL blocked.";

        return View("Index");
    }
}

```

4. Index.cshtml

```

<h1>ASP.NET Core Security Demo</h1>

<hr />

```

<h2>1. XSS Attack and Prevention</h2>

```
<form asp-action="Xss" method="post">
```

```
    <input name="name"
           placeholder="Enter name" />
```

```
    <button type="submit">
        Submit
    </button>
```

```
</form>
```

```
@if (ViewBag.XssResult != null)
{
```

```
    <p>
        Output:
        @ViewBag.XssResult
    </p>
}
```

```
<p>
    Test input:
    <code>&lt;script&gt;alert('XSS')&lt;/script&gt;</code>
</p>
```

```
<hr />
```

<h2>2. SQL Injection</h2>

<h3>Unsafe Login</h3>

```
<form asp-action="UnsafeLogin" method="post">
```

```
    <input name="username"
           placeholder="Username" />
```

```
    <input name="password"
           placeholder="Password" />
```

```
    <button type="submit">
        Unsafe Login
    </button>
```

```
</form>
```

<h3>Safe Login</h3>

```
<form asp-action="SafeLogin" method="post">
```

```
    <input name="username"
           placeholder="Username" />
```

```
    <input name="password"
           placeholder="Password" />
```

```
    <button type="submit">
        Safe Login
    </button>
```

```

</form>

@if (ViewBag.SqlResult != null)
{
    <p>@ViewBag.SqlResult</p>
}

<p>
    SQL Injection test:
    <code>' OR '1'='1</code>
</p>

<hr />

<h2>3. CSRF Protection</h2>

<form asp-action="Submit" method="post">
    @Html.AntiForgeryToken()

    <input name="message"
        placeholder="Enter message" />

    <button type="submit">
        Submit
    </button>

</form>

@if (ViewBag.CsrfResult != null)
{
    <p>@ViewBag.CsrfResult</p>
}

<hr />

<h2>4. Open Redirect</h2>

<h3>Unsafe Redirect</h3>

<a href="/Security/UnsafeRedirect?url=https://example.com">
    Test Unsafe Redirect
</a>

<h3>Safe Redirect</h3>

<a href="/Security/SafeRedirect?url=/Security">
    Test Safe Local Redirect
</a>

@if (ViewBag.RedirectResult != null)
{
    <p>@ViewBag.RedirectResult</p>
}

```

OUTPUT:

JavaScript is not executed.

1. XSS Attack and Prevention

Output: <script>alert('XSS')</script>

Test input: <script>alert('XSS')</script>

Unsafe login (SQL Injection)

2. SQL Injection

Unsafe Login

Safe Login

Login successful (unsafe query).

SQL Injection Prevented

2. SQL Injection

Unsafe Login

Safe Login

Invalid login.

CSRF Token Passed.

3. CSRF Protection

Request accepted. Anti-forgery token is valid.

Open Redirect Vulnerability (redirects to external website)

4. Open Redirect

Unsafe Redirect

[Test Unsafe Redirect](#)

Safe Redirect

[Test Safe Local Redirect](#)

<https://localhost:44333/Security/UnsafeRedirect?url=https://example.com>

External Redirection Blocked

[Test Safe Local Redirect](#)

External URL blocked.

© 2026 - SecurityDemo - [Privacy](#)

<https://localhost:44333/Security/SafeRedirect?url=https://example.com>

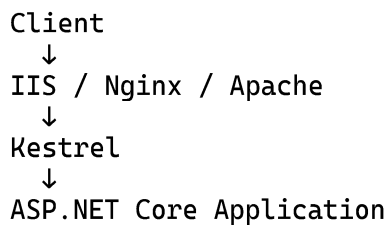
Unit – 9

1. Hosting and Deploying ASP.NET Core Application

1. Theory

ASP.NET Core Hosting

Hosting means running an ASP.NET Core application on a server so that users can access it. Kestrel is the default web server of ASP.NET Core. In production, Kestrel is commonly used behind a reverse proxy such as IIS, Nginx, or Apache.



Hosting Models

Server	Description
IIS	Microsoft's web server for Windows and supports ASP.NET Core through ANCM.
Nginx	High-performance web server, commonly used as a reverse proxy on Linux.
Apache	Cross-platform web server that can work as a reverse proxy for Kestrel.
Kestrel	Default, cross-platform ASP.NET Core web server.

ASP.NET Core Module

The **ASP.NET Core Module** allows IIS to host ASP.NET Core applications. IIS receives the request and ANCM forwards it to the ASP.NET Core application running through Kestrel.

Client → IIS → ANCM → Kestrel → ASP.NET Core

2. Deployment Procedure

Step 1: Build

```
dotnet build -c Release
```

Step 2: Publish

In Visual Studio:

1. Right-click the project in **Solution Explorer**.
2. Select **Publish**.

3. Select **Folder**.
4. Select **Release** configuration.
5. Click **Publish**.

The published files are generated in a folder such as:

bin\Release\net10.0\publish\

Step 3: Deploy and Run

- Create C:\inetpub\SecurityDemo
- Paste content from bin\Release\net10.0\publish\
- Install IIS Server
- Open Inet Manager
Win + R -> inetmgr
- Add application pool
- Add website
- Run the website

3. Docker Containerization

Docker packages the application and its required runtime into a container.

ASP.NET Core

↓

Dockerfile

↓

Docker Image

↓

Container

↓

Kestrel

Build Dockerfile:

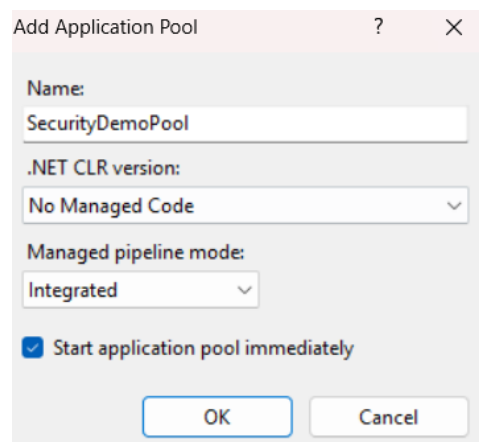
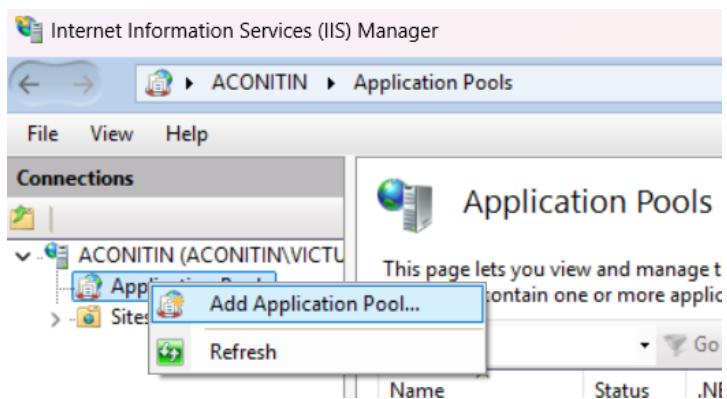
```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish -c Release -o /app/publish
```

```
FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
COPY --from=build /app/publish .
ENTRYPOINT ["dotnet", "SecurityDemo.dll"]
```

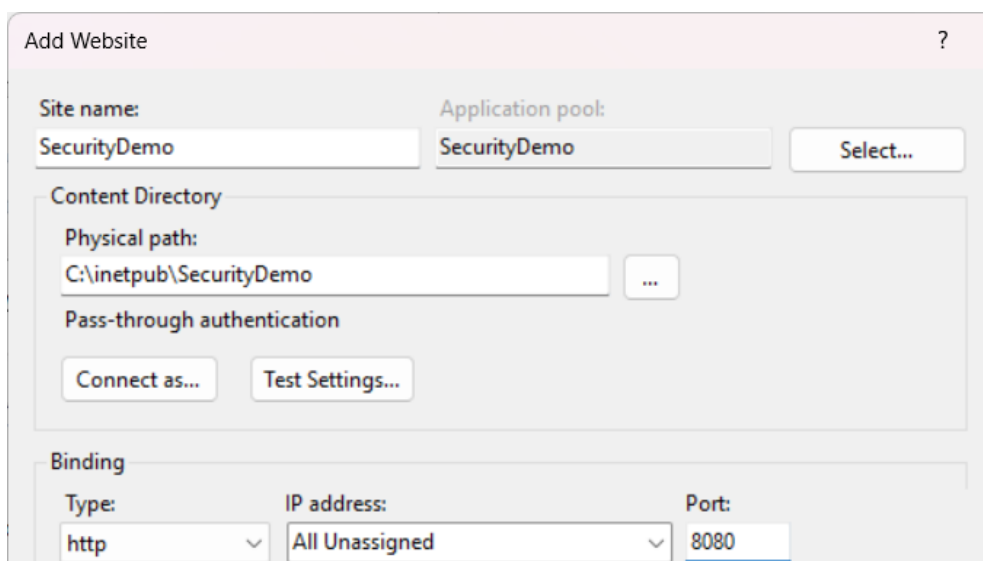
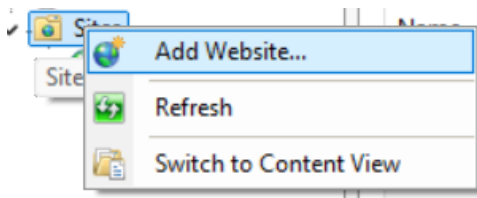
Build and run:

```
docker build -t securitydemo .
docker run -p 8080:8080 securitydemo
```

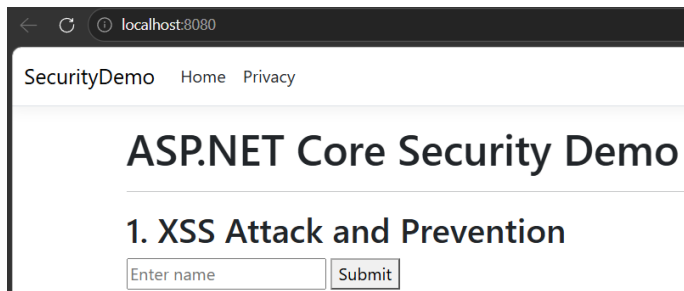

Add Application Pool



Add Website



Accessing Website Hosted on IIS

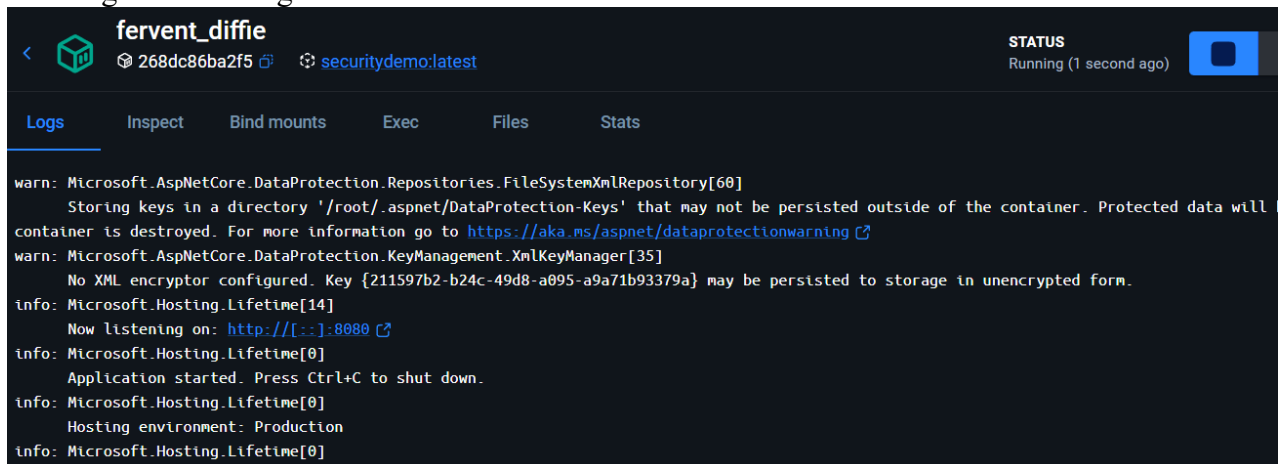


Publish using Docker

Building Docker Image

```
D:\Online Class\suresh\SecurityDemo\SecurityDemo>docker build -t securitydemo .
[+] Building 8.1s (4/12)
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 291B
=> [internal] load metadata for mcr.microsoft.com/dotnet/sdk:8.0
=> [internal] load metadata for mcr.microsoft.com/dotnet/aspnet:8.0
=> [internal] load .dockerignore
=> => transferring context: 2A
=> [internal] load build context
=> => transferring context: 82.21MB
=> [build 1/4] FROM mcr.microsoft.com/dotnet/sdk:8.0@sha256:bb32ba3ba3ea36e38572d9d8db76fa15f7cbf722f3f886e06bca
=> => resolve mcr.microsoft.com/dotnet/sdk:8.0@sha256:bb32ba3ba3ea36e38572d9d8db76fa15f7cbf722f3f886e06bca6d528b
=> => sha256:a5731be9f2ef507f77c1ee99f904922c863db51aff682e67e94ee2c58af6a78 0B / 17.05MB
=> [stage-1 1/3] FROM mcr.microsoft.com/dotnet/aspnet:8.0@sha256:787c228ea85457bec43c8b084de6ac360b26ea43b5c2fcbe
=> => resolve mcr.microsoft.com/dotnet/aspnet:8.0@sha256:787c228ea85457bec43c8b084de6ac360b26ea43b5c2fcbe861f721f
=> => sha256:6a13df20059f818fcb473629156686d32c95417b08e508ffe6023df42ae174 11.10MB / 11.10MB
=> => sha256:95f74afdb0f5ca9cd3c0cd4542c068b7c3dd2f882375c23d2eb54806163c1b8d 165B / 165B
=> => sha256:7b80194f63c035bedd5b4c27099bd45bd8ffdd8d160fe232f60a168c35b69d87 5.24MB / 32.26MB
=> => sha256:241824c35c3fd308c89de22df72bc5c829022499abc318ceacfc8d8105467cd1 3.34kB / 3.34kB
=> => sha256:de2662b08655c8b67db5858cb8a7ba0ec8141bc23773ce5b55868e4aa878c0d 4.19MB / 18.74MB
=> => sha256:a8ac7f6c6756c236e4c745052c404112b8fab6e8ac3a329d1ef3b867ad67c71 7.34MB / 28.23MB
```

Running Docker Image



Visiting Website Through Docker

